# Large Language Models for Designing Participatory Budgeting Rules

Nguyen Thach **Xingchen Sha** Hau Chan

Research Interests: Preference Aggregation, Mechanism Design, and AI Alignment.

Contact: Xingchen.sha@u.northwestern.edu

# Motivation

- Participatory budgeting (PB) selects public projects under a budget constraint.
- We want **high utility** and **good fairness**, but existing rules usually favor one side.
- Handcrafted PB rules require significant domain expertise.
- Large language models (LLMs) have shown remarkable capabilities in **algorithmic design**.

**Core question**

Can we design PB rules that are empirically strong on a utility objective while remaining approximately fair?

# Main Technical Ideas

- **Challenge 1:** PB is a multi-objective problem, utility and fairness.
- **Challenge 2:** Fairness is hard to measure during search.
- **Challenge 3:** Verifying standard fairness notions can be computationally expensive.

- We introduce **Strong-EJR approximation** $\phi \in [0, 1]$ as a fairness objective.
- It is basically a normalized average representation ratio over deserving cohesive groups.

# The LLMRule Framework

- Maintain a population of heuristic PB rules.
- Each rule has
  - a natural-language description
  - executable code
  - a fitness score
- The LLM iteratively
  - proposes new rules
  - modifies promising rules
  - refines prompt strategies when search stagnates

$$q(f) = \frac{1}{K} \sum_{k=1}^{K} \left[ \omega'(I^{(k)}, A^{(k)}, f) - \theta\big(\phi(I^{(k)}, A^{(k)}, f)\big) \right]$$

- Intuition: Reward relative welfare & Penalize rules that are not empirically fair.

# Experimental Setup

- **617** real-world PB instances from Pabulib
- Data from the **U.S., Canada, Poland, and the Netherlands**
- Three settings
  - approval ballots with cost satisfaction
  - approval ballots with cardinality satisfaction
  - cardinal ballots
- Train on smaller instances, then test on larger instances
- Baselines include **GreedUtil**, **MES**, **SeqPhrag**, and **MaximinSupp**

# Results on Approval Instances

| Setting | Rule | Welfare $\omega'$ | Fairness $\phi$ |
|---|---|---|---|
| $\mathrm{sat}_{cost}$ | GreedUtil | 0.971 | 0.686 |
| $\mathrm{sat}_{cost}$ | SeqPhrag | 0.695 | 0.956 |
| $\mathrm{sat}_{cost}$ | LLMRule | **0.814** | **0.901** |
| $\mathrm{sat}_{card}$ | SeqPhrag | 0.959 | 0.960 |
| $\mathrm{sat}_{card}$ | MaximinSupp | 0.977 | 0.964 |
| $\mathrm{sat}_{card}$ | LLMRule | **0.986** | 0.941 |

- LLMRule improves utility over proportional baselines.
- For $\mathrm{sat}_{cost}$, it also improves fairness over GreedUtil.
- The good behavior also extends to the cardinal-ballot setting.

# Takeaways

- **LLMRule** is the first LLM-based framework for designing participatory budgeting rules.
- Empirically, the learned rules achieve strong utility-fairness tradeoffs on real PB data.
- More broadly, this is a promising step toward **automated mechanism design** with LLMs.

**Rule Description (generated using the initial M1 prompt strategy)**

The *new* strategy modifies the scoring function to emphasize projects with high approval rates while proportionally reducing the influence of costly projects, using a linear budget fraction combined with a logarithmic transformation of approval rates.

```python
import numpy as np

def priority(project_costs, budget, approval_mat):
    '''
    Args:
    project_costs (np.ndarray): Array of length M storing the project costs.
    budget (int or float): A positive value.
    approval_mat (np.ndarray): An N-by-M matrix where each row contains a sample of binary approvals for the respective
        projects.

    Returns:
    scores (np.ndarray): Array of priority scores for the projects.
    '''

    # Calculate the average approval rates
    avg_approvals = np.mean(approval_mat, axis=0)

    # Calculate budget fractions
    budget_fractions = budget / project_costs

    # Calculate scores based on linear budget fractions and logarithmic approval rates
    scores = avg_approvals * (budget_fractions) * np.log(1 * avg_approvals)

    return scores
```

**Figure S6: High-level description of an LLM-generated PB rule for approval ballots with $sat = sat^{avd}$ and its associated code.**

```python
import numpy as np

def priority(project_costs, budget, valuation_mat):
    '''
    Args:
    project_costs (np.ndarray): Array of length M storing the project costs.
    budget (int or float): A positive value.
    valuation_mat (np.ndarray): An N-by-M matrix where each row contains a sample of valuations for the respective
        projects.

    Returns:
    scores (np.ndarray): Array of priority scores for the projects.
    '''

    M = len(project_costs)
    N = valuation_mat.shape[0]
    scores = np.zeros(M)

    for i in range(M):
        average_valuation = np.mean(valuation_mat[:, i])
        cost_efficiency = (budget / project_costs[i]) if project_costs[i] <= budget else 0
        normalized_cost_penalty = 1 - (project_costs[i] / budget) if project_costs[i] <= budget else 0

        # Calculate score
        scores[i] = (average_valuation * cost_efficiency) - normalized_cost_penalty

    # Ensure top N projects based on average valuation are included
    top_n_indices = np.argsort(-np.mean(valuation_mat, axis=0))[:N]
    for idx in top_n_indices:
        if scores[idx] < 0:
            scores[idx] = 0  # Ensure the score is non-negative

    return scores
```

**Figure S7: High-level description of an LLM-generated PB rule for cardinal ballots and its associated code.**

Thank you